

<u>CL-2006</u> <u>Operating System Lab</u>	<u>LAB -06</u> <u>CPU Scheduling Algorithms</u>
---	--

National University of Computer and Emerging Science
Spring 2026

Table of Content

Objective.....	3
CPU Scheduling.....	3
Key Metrics Calculated:.....	3
CPU Scheduling Algorithms.....	3
1. First Come First Serve (FCFS):.....	3
2. Round Robin with varying time quantum (RR):.....	4
3. Shortest Process Next (SPN): Shortest Job First (non-preemptive).....	5
4. Shortest Remaining Time (SRT): Shortest Remaining Time First (non-preemptive).....	5
5. Feedback (FB): Multi-Level Queue.....	6
6. Feedback with varying time quantum (FBV): Multi-Level Feedback queue.....	7
Cloning a GITHUB Repository in Bash Shell:.....	7
Input Format of test cases.....	9
Lab Activity.....	10

Objective

After completing this lab, you will be able to:

- Understand the working of scheduling algorithms through test scenarios. Calculation of average response, waiting and turnaround time for each case.
- Understanding the test-case generation format
- Run all test cases for each algorithm and compute by hand average response, waiting and turnaround times based on each test case output.
- Make two test cases on your own for trace and stats.

CPU Scheduling

CPU scheduling determines which process gets to use the CPU at any given time. The operating system maintains a queue of processes waiting for CPU time and uses scheduling algorithms to decide the order.

Key Metrics Calculated:

1. **Arrival Time (AT):** When a process enters the system.
2. **Service Time (ST) / Burst Time:** How long the process needs CPU time.
3. **Waiting Time (WT):** Time a process spends waiting in the queue before execution.

$$WT = \text{Start Time} - \text{Arrival Time}$$

$$\text{Average Waiting Time} = \text{Sum of all WT} / \text{Number of Processes}$$

4. **Turnaround Time (TAT):** Total time from arrival to completion.

$$TAT = \text{Completion Time} - \text{Arrival Time} / TAT = WT + ST$$

$$\text{Average Turnaround Time} = \text{Sum of all TAT} / \text{Number of Processes}$$

5. **Response Time (RT):** Time from arrival to first execution.

$$RT = (\text{Time of First Execution}) - (\text{Arrival Time})$$

$$\text{Average Response Time} = \text{Sum of all RT} / \text{Number of Processes}$$

CPU Scheduling Algorithms

1. First Come First Serve (FCFS):

First Come First Served (FCFS) is a scheduling algorithm in which the process that arrives first is executed first. It is a simple and easy-to-understand algorithm, but it can lead to poor performance if there are processes with long burst times. This algorithm does not have any mechanism for prioritizing processes, so it is considered a non-preemptive algorithm.

In FCFS scheduling, the process that arrives first is executed first, regardless of its burst time or priority. This can lead to poor performance, as longer running processes will block shorter ones from being executed. It is commonly used in batch systems where the order of the processes is important.

Example: Processes arriving:

A(arrives at 0, needs 3 time units),

B(arrives at 2, needs 6 time units),

C(arrives at 4, needs 4 time units)

Timeline:

Time:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
CPU:	A	A	A	B	B	B	B	B	B	C	C	C	C	-

A: Starts at 0, finishes at 3

B: Waits from 2-3, runs 3-9 (WT=1, TAT=7)

C: Waits from 4-9, runs 9-13 (WT=5, TAT=9)

2. Round Robin with varying time quantum (RR):

- Round Robin (RR) with variable quantum is a scheduling algorithm that uses a time-sharing approach to divide CPU time among processes. In this version of RR, the quantum (time slice) is not fixed and can be adjusted depending on the requirements of the processes. This allows processes with shorter burst times to be given smaller quanta and vice versa.
- The algorithm works by maintaining a queue of processes, where each process is given a quantum of time to execute on the CPU. When a process's quantum expires, it is moved to the back of the queue, and the next process in the queue is given a quantum of time to execute.
- The variable quantum allows the algorithm to be more efficient as it allows the CPU to spend more time on shorter processes and less time on longer ones. This can help to minimize the average waiting time for processes. Additionally, it also helps to avoid the issue of starvation, which occurs when a process with a long burst time prevents other processes from executing.

Example: With quantum 4, Processes arriving:

A(arrives at 0, needs 3 time units),

B(arrives at 2, needs 6 time units),

C(arrives at 4, needs 4 time units)

Timeline:

Time:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
CPU:	A	A	A	B	B	B	B	C	C	C	C	B	B	-

A: Runs 0-3, finishes (RT=0, WT=0, TAT=3)

B: Arrives 2, runs 3-7 (one quantum), then 12-14 (finishes) (RT=1, WT=5, TAT=12)

C: Arrives 4, runs 7-11 (one quantum), finishes (RT=3, WT=3, TAT=7)

3. Shortest Process Next (SPN): Shortest Job First (non-preemptive)

- Shortest Process Next (SPN) is a scheduling algorithm that prioritizes the execution of processes based on their burst time, or the amount of time they need to complete their task. It is a non-preemptive algorithm which means that once a process starts executing, it runs until completion or until it enters a waiting state.
- The algorithm maintains a queue of processes, where each process is given a burst time when it arrives. The process with the shortest burst time is executed first, and as new processes arrive, they are added to the queue and sorted based on their burst time. The process with the shortest burst time will always be at the front of the queue, and thus will always be executed next.
- This algorithm can be beneficial in situations where the objective is to minimize the average waiting time for processes, since shorter processes will be executed first, and thus will spend less time waiting in the queue. However, it can lead to longer running processes being blocked by shorter ones, which can negatively impact the overall performance of the system.
- In summary, SPN is a scheduling algorithm that prioritizes the execution of processes based on their burst time, it's non-preemptive and it's commonly used in situations where the objective is to minimize the average waiting time for processes.

Example: Processes arriving:

A(arrives at 0, needs 3 time units),

B(arrives at 2, needs 6 time units),

C(arrives at 4, needs 4 time units)

Timeline:

Time:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
CPU:	A	A	A	C	C	C	C	B	B	B	B	B	B	-

A: Starts at 0, finishes at 3 (WT=0, TAT=3)

C: Arrives 4, but waits for A. Runs 3-7 (WT=-1+3=0, TAT=3)

B: Arrives 2, waits. Runs 7-13 (WT=5, TAT=11)

4. Shortest Remaining Time (SRT): Shortest Remaining Time First (non-preemptive)

- Shortest Remaining Time Next (SRT) is a scheduling algorithm that is similar to the Shortest Process Next (SPN) algorithm, but it is a preemptive algorithm. This means that

once a process starts executing, it can be interrupted by a new process with a shorter remaining time.

- The algorithm maintains a queue of processes, where each process is given a burst time when it arrives. The process with the shortest remaining time is executed first, and as new processes arrive, they are added to the queue and sorted based on their remaining time. The process with the shortest remaining time will always be at the front of the queue, and thus will always be executed next.
- This algorithm can be beneficial in situations where the objective is to minimize the average waiting time for processes, since shorter processes will be executed first, and thus will spend less time waiting in the queue. Additionally, it can be useful in situations where the burst time of processes is not known in advance, since the algorithm can adapt to changes in the remaining time as the process is executing.
- In summary, SRT is a scheduling algorithm that prioritizes the execution of processes based on their remaining time, it's a preemptive algorithm, which means that it can interrupt a process that's already executing if a new process with a shorter remaining time arrives and it's commonly used in situations where the objective is to minimize the average waiting time for processes and burst time is not known in advance.

Example: Processes arriving:

A(arrives at 0, needs 3 time units),

B(arrives at 2, needs 6 time units),

C(arrives at 4, needs 4 time units)

Timeline:

Time:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
CPU:	A	A	B	C	C	C	C	B	B	B	B	B	A	-

When C arrives at time 4:

- A has 1 time unit remaining
- B has 5 time units remaining
- C has 4 time units remaining

C has the shortest remaining time, so it runs first.

5. Feedback (FB): Multi-Level Queue

- Feedback is a scheduling algorithm that allocates CPU time to different processes based on their priority level. It is a multi-level priority algorithm that uses multiple priority queues, each with a different priority level.
- Processes with higher priority levels are executed first, and as new processes arrive, they are added to the appropriate priority queue based on their priority level. When a process completes execution, it is moved to the next lower priority queue.
- This algorithm can be beneficial in situations where the system needs to handle a mix of short and long-running processes, as well as processes with varying priority levels. By

having multiple priority queues, it ensures that processes with higher priority levels are executed first, while also allowing lower-priority processes to eventually be executed.

- In summary, Feedback is a scheduling algorithm that allocates CPU time based on priority levels, it uses multiple priority queues with different levels of priority, processes with higher priority levels are executed first and when process completes execution, it is moved to the next lower priority queue, it's commonly used in situations where system needs to handle a mix of short and long - running processes, as well as processes with varying priority levels.

Example:

Priority Queue 1: quantum = 1

Priority Queue 2: quantum = 2

Priority Queue 3: quantum = 4

Priority Queue 4: quantum = 8

... and so on

B-1 means all queues have quantum = 1

FB-2ⁱ means queue i has quantum = 2ⁱ

Process A (needs 3 time units):

- Runs 1 unit in queue 1, moves to queue 2
 - Runs 1 unit in queue 2, moves to queue 3
 - Runs 1 unit in queue 3, finishes
- Total time = 3

Process B (needs 10 time units):

- Runs 1 unit in queue 1, moves to queue 2
- Runs 2 units in queue 2, moves to queue 3
- Runs 4 units in queue 3, moves to queue 4
- Runs 3 units in queue 4, finishes

6. Feedback with varying time quantum (FBV): **Multi-Level Feedback queue**

- Same as Feedback but with varying time quantum.
- Feedback with varying time quantum also uses multiple priority queues and assigns a different time quantum for each priority level, it allows the algorithm to be more efficient by spending more time on higher-priority processes and less time on lower-priority processes.

Cloning a GITHUB Repository in Bash Shell:

1. Copy the URL for the repository.
2. Open Bash Shell

3. Change the current working directory to the location where you want the cloned directory.
4. Type git clone, and then paste the URL mentioned below.
<https://github.com/yousefktop/CPU-Scheduling-Algorithms.git>
5. Press Enter to create your local clone.

```
nakafi@ubuntu:~$  
nakafi@ubuntu:~$ git clone https://github.com/yousefktop/CPU-Scheduling-Algorithms.git  
Cloning into 'CPU-Scheduling-Algorithms'...  
remote: Enumerating objects: 252, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Compressing objects: 100% (2/2), done.  
remote: Total 252 (delta 2), reused 1 (delta 1), pack-reused 249  
Receiving objects: 100% (252/252), 396.13 KiB | 516.00 KiB/s, done.  
Resolving deltas: 100% (145/145), done.  
nakafi@ubuntu:~$
```

Compile and execute the code using makefile to generate the executable file.

```
nakafi@ubuntu:~/CPU-Scheduling-Algorithms$ pwd  
/home/nakafi/CPU-Scheduling-Algorithms  
nakafi@ubuntu:~/CPU-Scheduling-Algorithms$  
nakafi@ubuntu:~/CPU-Scheduling-Algorithms$  
nakafi@ubuntu:~/CPU-Scheduling-Algorithms$ make  
g++ -c main.cpp  
g++ -o lab4 main.o  
nakafi@ubuntu:~/CPU-Scheduling-Algorithms$ cat makefile  
target:  
    g++ -c main.cpp  
    g++ -o lab4 main.o  
nakafi@ubuntu:~/CPU-Scheduling-Algorithms$ █
```

Read the input files to identify the algorithms input sequence:

```
nakafi@ubuntu:~/CPU-Scheduling-Algorithms/testcases$ cat 01a-input.txt  
trace  
1  
20  
5  
A,0,3  
B,2,6  
C,4,4  
D,6,5  
E,8,2
```


Run the code to analyze the working of different CPU scheduling algorithms depending on the input.

```

nakaf1@ubuntu:~/CPU-Scheduling-Algorithms$ ./lab4
trace
1
20
5
A,0,3
B,2,6
C,4,4
D,6,5
E,8,2
FCFS  0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0
-----
A      |*|*|*| | | | | | | | | | | | | | | | |
B      | | | .|*|*|*|*|*| | | | | | | | | | |
C      | | | | | .|.|.|.|.|.|*|*|*|*| | | | |
D      | | | | | | | .|.|.|.|.|.|.|.|*|*|*|*|*|
E      | | | | | | | | | .|.|.|.|.|.|.|.|.|.|.|*|*|
-----

```

Input Format of test cases

Line 1: "trace" or "stats"

Line 2: a comma-separated list telling which CPU scheduling policies to be analyzed/visualized along with their parameters, if applicable. Each algorithm is represented by a number as listed in the introduction section and as shown in the attached test cases. Round Robin and Aging have a parameter specifying the quantum q to be used. Therefore, a policy entered as 2-4 means Round Robin with q=4. Also, policy 8-1 means Aging with q=1.

1. FCFS (First Come First Serve)
2. RR (Round Robin)
3. SPN (Shortest Process Next)
4. SRT (Shortest Remaining Time)
5. FB-1, (Feedback where all queues have q=1)
6. FB-2i, (Feedback where q= 2i)

trace	stats
1	1
20	20
5	5
A,0,3	A,0,3
B,2,6	B,2,6
C,4,4	C,4,4
D,6,5	D,6,5
E,8,2	E,8,2
(a)	(b)

Line 3: An integer specifying the last instant to be used in your simulation and to be shown on the timeline.

Line 4: An integer specifying the number of processes to be simulated.

Line 5: Start of description of processes. Each process is to be described on a separate line. For algorithms 1 through 7, each process is described using a comma-separated list specifying:

1. String specifying a process name
2. Arrival Time
3. Service Time

Processes are assumed to be sorted based on the arrival time. If two processes have the same arrival time, then the one with the lower priority is assumed to arrive first.

Lab Activity

1. Complete the installation.
2. Compile and execute the output file using the testcase shown above.
3. Read "Input Format of test cases" and verify it with file 01a-input.txt and 01b-input.txt shown in the Figure 1 part (a) and (b).
4. Now run all test cases and calculate the values by hand.
5. Submit snapshots of your hand-written work on GCR and hardcopy during the lab.